STUDIENPROJEKT I

Introduction to Homomorphic Encryption - The BFV Scheme

Tim Widmoser

Matrikelnummer: 77204787039

Fachbereich 2

Duales Studium Informatik - Jahrgang 2024

Datum d. Fertigstellung: 22.06.2026

Wortanzahl: 4031

Abstract

As digital infrastructures expand, securing data across its entire lifecycle has become paramount. While traditional cryptographic schemes effectively protect data at rest and in transit, they require decryption prior to computation, creating critical vulnerabilities for "data in use." Homomorphic Encryption (HE) addresses this architectural flaw by enabling direct mathematical operations on ciphertext. This paper provides a comprehensive examination of the Brakerski/Fan-Vercauteren (BFV) scheme, a prominent scale-invariant homomorphic encryption system based on the Ring Learning With Errors (RLWE) problem. Through a step-by-step mathematical demonstration utilizing a simplified polynomial quotient ring, I detail the mechanics of key generation, encryption, and homomorphic evaluation (addition and multiplication). Furthermore, I analyze the inherent challenge of quadratic noise accumulation during ciphertext multiplication and discuss the necessity of bootstrapping to transition toward Fully Homomorphic Encryption (FHE). Finally, the paper explores the transformative potential of HE across highly regulated industries, including secure cloud computing, privacy-preserving healthcare analytics, and financial fraud detection, highlighting its emerging role as a foundational technology for zero-trust data privacy.

Keywords: Homomorphic Encryption; Cryptography; Somewhat Homomorphic Encryption; BFV; Algebra; Computer Science; Mathematics

Contents

1. Declaration of Originality 2
2. Cryptography 3
3. Homomorphic Encryption 5
4. Homomorphic Schemes 6
5. Preliminaries 7
6. The Somewhat Homomorphic BFV Scheme 8
7. Necessity of Bootstrapping 16
8. Industry Usage 17
9. Conclusion 19

References 20

Citations and references displayed in APA style: (Fan & Vercauteren, 2012)

1. Declaration of Originality

I hereby declare that I have completed the present work independently in all parts and have not used any sources or aids other than those indicated in the work. All verbatim or paraphrased passages and quotations, as well as all sections that were designed, written, and/or revised with the help of AI-based tools, are clearly identified and documented. In the appendix of my work, I have listed all AI-based tools used. These are documented in an AI directory, including product names and the corresponding prompts used. I am aware that the use of texts or other content and products generated by AI-based tools does not guarantee their quality. I take full responsibility for the inclusion of any machine-generated passages I have used and bear responsibility for any potentially incorrect or biased content generated by AI, inaccurate references, violations of data protection or copyright law, or plagiarism.

1.1. Eigenständigkeitserklärung

„Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Sämtliche wörtlichen oder sinngemäßen Übernahmen und Zitate, sowie alle Abschnitte, die mithilfe von KI-basierten Tools entworfen, verfasst und/oder bearbeitet wurden, sind kenntlich gemacht und nachgewiesen. Im Anhang meiner Arbeit habe ich sämtliche KI-basierte Hilfsmittel angegeben. Diese sind mit Produktnamen und formulierten Eingaben (Prompts) in einem KI-Verzeichnis ausgewiesen. Ich bin mir bewusst, dass die Verwendung von Texten oder anderen Inhalten und Produkten, die durch KI-basierte Tools generiert wurden, keine Garantie für deren Qualität darstellt. Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.“

Place, Date: _____

Signature: _____

2. Cryptography

2.1. Introduction

With the rapid expansion of global digitization, nearly every facet of modern human life has transitioned into the online realm. From routine communication, social interactions, and digital entertainment to critical infrastructure, healthcare diagnostics, governmental services, and financial transactions, the volume of data generated, transmitted, and stored on a daily basis has reached an unprecedented scale. Consequently, a person's most sensitive information is now persistently distributed across various servers, third-party cloud architectures, institutional platforms, and interconnected databases. This paradigm shift has unlocked tremendous technological, social, and economic potential by enabling faster communication, automated decision-making, and large-scale data analysis. However, it has simultaneously exposed individuals, corporations, and public institutions to severe privacy risks, unauthorized surveillance, and malicious exploitation. As data has become one of the most valuable commodities of the 21st century, the demand for secure online services has grown exponentially. In this context, the emergence, refinement, and continuous evolution of cryptography have become not merely beneficial, but an absolute necessity for maintaining trust in digital environments.

To mitigate the inherent vulnerabilities of an interconnected world, cryptographic protocols have become the foundational layer of digital trust. Modern cryptography extends far beyond its historical roots of simple message obfuscation or secret communication between limited parties. Instead, it can be formally described as the "study of mathematical techniques for securing digital information, systems, and distributed computations against adversarial attacks" (Katz & Lindell, 2015). Traditionally, these mathematical techniques have been highly successful in establishing secure communication channels, authenticating users, verifying data integrity, and protecting stored information from unauthorized access. Through the rigorous application of symmetric and asymmetric encryption algorithms, as well as secure key-exchange mechanisms and digital signatures, the industry has largely solved the challenges of securing data "at rest" on physical hardware and data "in transit" across public or untrusted networks. These achievements have enabled the modern internet to function as a reliable platform for commerce, communication, and institutional operations.

Despite these monumental successes, a critical vulnerability remains within the classical cryptographic architecture. In traditional systems, whenever data needs to be processed, analyzed, searched, or queried by a computer, it must first be decrypted back into plaintext within the hardware's memory or processing environment. This temporary return to an unencrypted state creates a highly vulnerable attack vector, exposing raw data to compromised cloud environments, insider threats, malware, memory scraping attacks, and increasingly sophisticated side-channel attacks. The problem becomes especially significant in outsourced computation scenarios, where users and organizations rely on external cloud providers to store and process sensitive information. In such environments, encryption may protect data before and after computation, but not during the computational process itself. Addressing this fundamental architectural flaw, namely the inability to secure "data in use," therefore requires a profound shift in cryptographic design. This challenge has paved the way for revolutionary concepts that allow meaningful operations to be performed directly on actively encrypted information, without requiring exposure of the underlying plaintext.

2.2. Types of Cryptographic Schemes

To understand the paradigm shift introduced by homomorphic encryption, it is essential to first categorize classical cryptographic systems and understand their fundamental limitations. Traditional schemes are broadly classified into two domains: symmetric and asymmetric cryptography

2.2.1. Symmetric Cryptography

In a symmetric-key scheme, such as the Advanced Encryption Standard (AES), the same cryptographic key is used for both the encryption of the plaintext and the decryption of the ciphertext. While computationally highly efficient and robust against many classical attacks, symmetric schemes suffer from the "key distribution problem"—the inherent risk of securely sharing the secret key between communicating parties over an untrusted channel

> **Example:**
>
> Encryption applies the key to the message:
>
> $$c = m \oplus k = 1011_2 \oplus 0110_2 = 1101_2$$
>
> Decryption applies the exact same key to the ciphertext to recover the message:
>
> $$m = c \oplus k = 1101_2 \oplus 0110_2 = 1011_2$$

2.2.2. Asymmetric Cryptography

To resolve the key distribution dilemma, asymmetric (or public-key) cryptography was introduced. Schemes like RSA and Elliptic Curve Cryptography (ECC) utilize a mathematically linked key pair: a public key used exclusively for encryption, and a private key retained by the owner for decryption. While asymmetric schemes solve the key distribution problem, they are computationally heavier than their symmetric counterparts.

> **Example:**
>
> In textbook RSA, choose two primes, $p = 3$ and $q = 11$. The modulus is $n = p \cdot q = 33$, and Euler's totient is $\varphi(n) = (p-1)(q-1) = 20$. Choose a public exponent $e = 3$. The private exponent d is computed as the modular inverse of e, such that $e \cdot d \equiv 1 (\mathrm{mod}\, \varphi(n))$, resulting in $d = 7$ (since $3 \cdot 7 = 21 \equiv 1$).
>
> For a plaintext message $m = 5$:
>
> Encryption uses only the public key (e, n):
>
> $$c = m^e (\mathrm{mod}\, n) = 5^3 (\mathrm{mod}\, 33) = 125 (\mathrm{mod}\, 33) = 26$$
>
> Decryption uses the private key (d, n):
>
> $$m = c^d (\mathrm{mod}\, n) = 26^7 (\mathrm{mod}\, 33) = 8031810176 (\mathrm{mod}\, 33) = 5$$

2.2.3. Vulnerabilities and the "Data in Use" Problem

Modern implementations of symmetric and asymmetric schemes are largely secure against brute-force and standard cryptanalytic attacks. However, they share a fundamental architectural limitation: they only secure data "at rest" (in storage) and "in transit" (over networks).

To perform any meaningful computation, statistical analysis, or database query, the ciphertext must be decrypted back into plaintext within the hardware's memory. This momentary decryption creates a critical attack vector, exposing the raw, unencrypted data to insider threats, memory-scraping malware, and side-channel attacks. This exact vulnerability, the inability to safely operate on "data in use", serves as the primary catalyst for the development of Homomorphic Encryption.

3. Homomorphic Encryption

In the past, operating on data required decrypting it, thereby making it vulnerable to attacks. Homomorphic Encryption allows for direct computations on the encrypted data. It follows the principles of a homomorphism and the process can broadly be written as follows:

$$\texttt{Dec}(\texttt{Enc}(x) \star \texttt{Enc}(y)) = x \star y$$

Its use cases span over several fields such as Cloud Computing, Machine Learning as well as the IoT. Performing statistical analyses or queries on sensitive data becomes possible without the loss of privacy. (Aaraj, 2022; Martins & others, 2017)

3.1. History of Homomorphic Schemes

The conceptual foundation of Homomorphic Encryption dates back to 1978, shortly after the invention of the RSA algorithm. Rivest, Adleman, and Dertouzos introduced the idea of "privacy homomorphisms," questioning whether it was possible to operate on encrypted data without first decrypting it. For decades, this remained an unsolved problem, though several Partially Homomorphic Encryption (PHE) schemes were developed. Notable examples include the RSA cryptosystem, which supports homomorphic multiplication, and the Paillier cryptosystem, which allows for homomorphic addition. However, a scheme capable of evaluating arbitrary circuits, both addition and multiplication simultaneously for an unlimited number of operations, eluded researchers.(Acar et al., 2017)

The pivotal breakthrough occurred in 2009 when Craig Gentry published his Ph.D. thesis, proposing the first Fully Homomorphic Encryption (FHE) scheme (Gentry, 2009). Gentry's construction relied on ideal lattices and introduced the revolutionary concept of "bootstrapping"—a mechanism to refresh ciphertexts by evaluating the decryption circuit homomorphically, thereby controlling the accumulated noise.

Gentry's work sparked rapid advancement in the field, leading to distinct "generations" of FHE schemes. The first generation was largely theoretical and computationally prohibitive. The second generation, emerging around 2011–2012, shifted the mathematical foundation to the Learning With Errors (LWE) and Ring Learning With Errors (RLWE) problems, which offered better security assumptions and significant performance improvements. It was during this period that the BGV (Brakerski-Gentry-Vaikuntanathan) and BFV (Brakerski/Fan-Vercauteren) schemes were introduced. The BFV scheme, specifically, offered scale-invariant properties, making noise management more predictable and implementation more practical.

Subsequent generations introduced further specializations. The third generation (e.g., GSW) focused on improving the bootstrapping process and conceptual simplicity, while the fourth generation, most notably the CKKS scheme , introduced approximate arithmetic for real and complex numbers, specifically tailored for machine learning applications. Today, the BFV scheme remains one of the most widely implemented standards for exact integer arithmetic within the homomorphic encryption landscape. (Marcolla et al., 2022)

4. Homomorphic Schemes

While the foundational concept of Homomorphic Encryption remains consistent—enabling computation on encrypted data without access to the secret key—the mathematical implementation and specific capabilities vary significantly depending on the underlying scheme. Since Gentry's initial breakthrough, several specialized FHE schemes have been developed, each optimized for different types of computational workloads and mathematical structures.

Modern homomorphic encryption schemes are generally categorized based on the type of arithmetic they support (exact vs. approximate) and the data types they naturally ingest. The most widely adopted schemes in research and industry today are BGV, BFV, CKKS, and TFHE.

Table 1. Overview of the most prominent Homomorphic Encryption schemes and their characteristics.

Scheme	Year	Data Type	Primary Use Case
BGV	2012	Integers / Fields	Smart contracts, database queries, and exact integer arithmetic.
BFV	2012	Integers / Fields	Scale-invariant arithmetic. Ideal for SIMD operations and exact statistics.
CKKS	2017	Real & Complex	Machine learning inference, where slight precision loss is acceptable.
TFHE	2016	Booleans	Fast evaluation of arbitrary logical boolean circuits (e.g., NAND, XOR).

4.1. Exact Integer Arithmetic (BGV and BFV)

The Brakerski-Gentry-Vaikuntanathan (BGV) and Brakerski/Fan-Vercauteren (BFV) schemes are designed to perform operations on integers modulo a plaintext prime t. Decrypting a ciphertext in these schemes yields the exact, mathematically precise result of the homomorphic operations, provided the accumulated noise budget has not exceeded the decryption threshold. While BGV manages noise by dynamically scaling the ciphertext modulus down after multiplications, BFV is scale-invariant, managing noise within a fixed ciphertext space. These schemes are highly effective for exact financial calculations, private set intersections, and precise statistical counts.

4.2. Approximate Continuous Arithmetic (CKKS)

The Cheon-Kim-Kim-Song (CKKS) scheme revolutionized the field by introducing homomorphic operations on real and complex numbers. To achieve this, CKKS intentionally treats encryption noise as part of the data's inherent approximation error. This makes its operation structurally similar to floating-point arithmetic in standard computer processors. Because it does not require absolute cryptographic exactness, CKKS has become the dominant scheme for training and evaluating Artificial Neural Networks (ANNs) on encrypted data, where overarching statistical trends are more important than absolute precision.

4.3. Boolean Circuit Evaluation (TFHE)

Alternatively, schemes like Fast Fully Homomorphic Encryption over the Torus (TFHE) take a completely different approach by evaluating individual boolean gates incredibly fast. Instead of operating on large vectors of integers or real numbers, TFHE allows for the homomorphic execution of any arbitrary binary circuit. Its highly optimized bootstrapping process takes only fractions of a second and can be performed after nearly every single gate operation, keeping the noise continuously low and allowing for infinite computational depth on binary structures.

5. Preliminaries

The following notations are important to remember throughout this work.

Modulo Representation

$$\mathbb{Z}_q = \mathbb{Z} \cap \left(-\frac{q}{2}, \frac{q}{2}\right]$$

This differs from the usual Ring defined under this notation.

5.1. Example

$\mathbb{Z}_{64} = \{-31, -30, ..., 0, ..., 32\}$

The Ring

Let $f(X)$ be a monic irreducible polynomial and $d = 2^n$. Define

$$\mathcal{R}_q = \mathbb{Z}_q / f(X)$$

5.2. Example

Let $\mathcal{R}_{32}, f(X) = x^2 + 1, A(X) = x^2 + x, B(X) = x^2 + 4x - 2$. Note that $f(X) = X^2 + 1 \Rightarrow x^2 \equiv -1$

Calculate $A(X) \cdot B(X)$:

$$\begin{aligned}(x^2 + x) \cdot (x^2 + 4x - 2) &\equiv (-1 + x) \cdot (-1 + 4x + 2) \\ &= (-1 + x) \cdot (4x + 1) \\ &= -4x + 4x^2 - 1 + x \\ &\equiv -4x + 4(-1) - 1 + x \\ &= -3x - 5\end{aligned}$$

Rounding

For $x \in \mathbb{R}$ let

$$\lfloor x \rceil = \lfloor x + 0.5 \rfloor$$

Applying the Modulo Ring

Let $a \in \mathcal{R}$ and let $[x]_q = x \bmod q$ such that $x \in \mathbb{Z}_q$.

$$[a]_q = \sum_{i=0}^{d-1} [a_i]_q x^i$$

5.3. Example

Let $q = 7, a = 10x^2 + 6x + 8$ with $\mathbb{Z}_7 = \{-3, .., 3\}$

$$a \equiv 3x^2 - x + 1$$

6. The Somewhat Homomorphic BFV Scheme

Secret Key Generation

Take in λ, sample $\boldsymbol{s} \xleftarrow{\chi} \mathcal{R}_2$ and output $\mathtt{sk} = \boldsymbol{s}$

Initializes the root of trust; coefficients of the secret key are typically bounded (usually ternary).

6.1. Preliminaries

Take $\mathbb{Z}[X]/(X^2+1)$, consider $q = 120, t = 12$, thus $\Delta = \frac{q}{t} = 10$. Let $\boldsymbol{s} = x + 1$.

Also let $\mathtt{pk} = (\boldsymbol{p_0}, \boldsymbol{p_1})$ with $\boldsymbol{p_1} = \boldsymbol{a} = 10x$ and $\boldsymbol{e} = 1$.

It follows from $\boldsymbol{p_0} = -(\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e})$ that $\boldsymbol{p_0} = -(10x \cdot (x+1) + 1) = -10x + 9$.

Remember

In $\mathbb{Z}[X]/(X^2+1)$ it follows that $x^2 \equiv -1$, thus we obtain

$$\begin{aligned}\boldsymbol{p_0} &= -(10x \cdot (x+1) + 1) \\ &= -(10x^2 - 10x + 1) \\ &= -(10 \cdot (-1) - 10x + 1) \\ &= -(-10x - 9) \\ &= -10x + 9\end{aligned}$$

Public Key Generation

Take in $\mathtt{sk}$, sample $\boldsymbol{a} \xleftarrow{\chi} \mathcal{R}_q$ and $\boldsymbol{e} \longleftarrow \chi$ and output:

$$\mathtt{pk} = \left([-(\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e})]_q, \boldsymbol{a}\right)$$

Generates the asymmetric keypair used for encrypting plaintext messages into the ciphertext space.

6.2. Example of generating a Public Key

Following from the above calculations, the public and secret key now have the following form:

$\texttt{sk} = x + 1 \qquad \texttt{pk} = ([-10x + 9], 10x)$

Key Generation Evaluation V1

Assume $p = q^k$ and χ' as a Discrete Gaussian with a variance $\sigma' > \sigma$ from χ. Take $(\texttt{sk}, p)$, sample $\boldsymbol{a} \xleftarrow{\chi} \mathcal{R}_{p \cdot q}$, $\boldsymbol{e} \longleftarrow \chi'$ and return:

$$\texttt{rlk} = \left(\left[-(\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e}) + p \cdot \boldsymbol{s}^2 \right]_{p \cdot q}, \boldsymbol{a} \right)$$

Produces the relinearization key necessary for reducing ciphertext dimensions after homomorphic multiplication.

6.3. Calculating the Relinearization Key

For simplicity we will not assume $p = q^k$, rather take $p = 100$. Therefore
$p \cdot s^2 = 100 \cdot 2x = 200x$.
Now sample $a \in \mathcal{R}_{100 \cdot 120}$. Let $a = 50$ and for simplicity $e = 0$.
Now the first part of relinearization key can be calculated:

$$\begin{aligned} -(\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e}) + p \cdot \boldsymbol{s}^2 &= -(50 \cdot (x + 1) + 0) + 200x \\ &= -50x - 50 + 200x \\ &= 150x - 50 \end{aligned}$$

Remember

For the secret key s, the following holds:

$$s^2 = (x + 1)^2 = x^2 + 2x + 1 \equiv -1 + 2x + 1 = 2x$$

Therefore the relinearization key

$\texttt{rlk} = ([150x - 50], 50)$

Encryption

Let $\boldsymbol{m} \in \mathcal{R}_t$, $\boldsymbol{p_0} = \texttt{pk}[0]$, $\boldsymbol{p_1} = \texttt{pk}[1]$, sample $\boldsymbol{u} \longleftarrow \mathcal{R}_2$, $\boldsymbol{e_1}, \boldsymbol{e_2} \longleftarrow \chi'$ and return:

$$\texttt{ct} = \left(\left[\boldsymbol{p_0} \cdot \boldsymbol{u} + \boldsymbol{e_1} + \Delta \cdot \boldsymbol{m} \right]_q, \left[\boldsymbol{p_1} \cdot \boldsymbol{u} + \boldsymbol{e_2} \right]_q \right)$$

Scales the plaintext message and masks it using the public key alongside fresh noise.

6.4. Encrypting the first message

Let $m_1 = 2$ and $m_2 = x$. Also let $u = -x, e_1 = 1 - x$ and $e_2 = -1$.
Now calculate the first entry of the ciphertext for m_1:

$$\begin{aligned}
\mathtt{ct}_1[0] &= [\boldsymbol{p_0} \cdot \boldsymbol{u} + \boldsymbol{e_1} + \Delta \cdot \boldsymbol{m}]_q \\
&= \underbrace{-10x + 9}_{p_0} \ \underbrace{-x}_{u} + \underbrace{(1 - x)}_{e_1} + \underbrace{10 \cdot 2}_{\Delta \cdot m} \\
&= 10x^2 - 9x + 1 - x + 20 \\
&\equiv -10 - 9x + 1 - x + 20 \\
&= -10x + 11
\end{aligned}$$

For the second entry of the ciphertext it follows:

$$\begin{aligned}
\mathtt{ct}_1[1] &= [\boldsymbol{p_1} \cdot \boldsymbol{u} + \boldsymbol{e_2}]_q \\
&= 10x \cdot (-x) + (-1) \\
&= -10x^2 - 1 \\
&\equiv -10 \cdot (-1) - 1 \\
&= 10 - 1 \\
&= 9
\end{aligned}$$

Thus the encryption of the first message $\mathtt{Enc}(m_1)$:

$$\mathtt{ct}_1 = ([-10x + 11], 9)$$

6.5. Encrypting the second message

For $m_2 = x$, sample a fresh $u = x + 1, e_1 = 2, e_2 = x - 1$

$$\begin{aligned}
\mathtt{ct}_2[0] &= [\boldsymbol{p_0} \cdot \boldsymbol{u} + \boldsymbol{e_1} + \Delta \cdot \boldsymbol{m}]_q \\
&= \underbrace{(-10x + 9)}_{p_0} \ \underbrace{(x + 1)}_{u} + \underbrace{2}_{e_1} + \underbrace{10 \cdot 2}_{\Delta \cdot m} \\
&= -10x^2 - 10x + 9x + 9 + 2 + 10x \\
&\equiv 10 - 10x + 9x + 9 + 2 + 10x \\
&= 10 + 9x + 9 + 2 \\
&= 9x + 21
\end{aligned}$$

For the second entry of the ciphertext it follows:

$$\begin{aligned}
\mathtt{ct}_2[1] &= [\boldsymbol{p_1} \cdot \boldsymbol{u} + \boldsymbol{e_2}]_q \\
&= (10x)(x + 1) + (x - 1) \\
&= 10x^2 + 10x + x - 1 \\
&\equiv -10 + 10x + x - 1 \\
&= -10 + 11x - 1 \\
&= 11x - 11
\end{aligned}$$

Thus the encryption of the second message $\mathtt{Enc}(m_2)$:

$$\mathtt{ct}_2 = ([9x + 21], [11x - 11])$$

Addition

Take $\mathtt{ct}_1$, $\mathtt{ct}_2$ and return:

$$\mathtt{ct}_+ = \Big([\mathtt{ct}_1[0] + \mathtt{ct}_2[0]]_q, [\mathtt{ct}_1[1] + \mathtt{ct}_2[1]]_q\Big)$$

Performs component-wise addition; the underlying plaintext messages are added implicitly.

6.6. Adding and encrypting two ciphertexts

$$\begin{aligned}
\mathtt{ct}_+ &= \Big([\mathtt{ct}_1[0] + \mathtt{ct}_2[0]]_q, [\mathtt{ct}_1[1] + \mathtt{ct}_2[1]]_q\Big) \\
&= ([[-10x + 11] + [9x + 21]]_{120}, [9 + [11x - 11]]_{120}) \\
&= (\underbrace{[-x + 32]}_{c_0}, \underbrace{[11x - 2]}_{c_1})
\end{aligned}$$

6.7. Decrypting the addition ciphertext

$$\begin{aligned}
\mathtt{Dec}(\mathtt{ct}_+) &= \left[\left\lfloor \frac{t \cdot [\boldsymbol{c_0} + \boldsymbol{c_1} \cdot \boldsymbol{s}]_q}{q} \right\rceil\right]_t \\
&= \left[\left\lfloor \frac{12 \cdot [(-x + 32) + (11x - 2) \cdot (x + 1)]_{120}}{120} \right\rceil\right]_{12} \\
&= \left[\left\lfloor \frac{12 \cdot [-x + 32 + 11x^2 + 11x - 2x - 2]_{120}}{120} \right\rceil\right]_{12} \\
&\equiv \left[\left\lfloor \frac{12 \cdot [-x + 32 + -11 + 11x - 2x - 2]_{120}}{120} \right\rceil\right]_{12} \\
&= \left[\left\lfloor \frac{12 \cdot [8x + 19]_{120}}{120} \right\rceil\right]_{12} \\
&= [\lfloor 0.1 \cdot (8x + 19) \rceil]_{12} \\
&= [\lfloor 0.8x + 1.9 \rceil]_{12} \\
&= [x + 2]_{12} \\
&= 2 + x \qquad \square
\end{aligned}$$

Note for the addition process

Since the messages to be added were $m_1 = 2 \wedge m_2 = x$, the expected result of $m_1 + m_2$ matches with the calculated result which is $2 + x$

Multiplication

Take $\texttt{ct}_1$, $\texttt{ct}_2$, $\texttt{rlk}$ and compute:

$$c_0 = \left[\left\lfloor\left(\frac{t \cdot (\texttt{ct}_1[0] \cdot \texttt{ct}_2[0])}{q}\right)\right\rceil\right]_q$$

$$c_1 = \left[\left\lfloor\left(\frac{t \cdot (\texttt{ct}_1[0] \cdot \texttt{ct}_2[1] + \texttt{ct}_1[1] \cdot \texttt{ct}_2[0])}{q}\right)\right\rceil\right]_q$$

$$c_2 = \left[\left\lfloor\left(\frac{t \cdot (\texttt{ct}_1[1] \cdot \texttt{ct}_2[1])}{q}\right)\right\rceil\right]_q$$

Computes the tensor product of the ciphertexts, temporarily expanding the ciphertext size to three elements.

To multiply two ciphertexts, c_0, c_1, c_2 have to be calculated:

Remember

$\texttt{ct}_1 = ([-10x + 11], 9), \texttt{ct}_2 = ([9x + 21], [11x - 11]), s = x + 1, \frac{t}{q} = \frac{12}{120} = 0.1$

6.8. Calculating c_0

$$\begin{aligned}
c_0 &= \left[\left\lfloor\left(\frac{t \cdot (\texttt{ct}_1[0] \cdot \texttt{ct}_2[0])}{q}\right)\right\rceil\right]_q \\
&= \left[\left\lfloor\frac{12 \cdot ((-10x + 11)(9x + 21))}{120}\right\rceil\right]_{120} \\
&= \left[\left\lfloor\frac{12 \cdot (-90x^2 - 210x + 99x + 231)}{120}\right\rceil\right]_{120} \\
&\equiv \left[\left\lfloor\frac{10 \cdot (90 - 210x + 99x + 231)}{120}\right\rceil\right]_{120} \\
&= \left[\left\lfloor\frac{12 \cdot (-111x + 321)}{120}\right\rceil\right]_{120} \\
&= [\lfloor 0.1 \cdot (-111x + 321)\rceil]_{120} \\
&= [\lfloor -11.1x + 32.1\rceil]_{120} \\
&= [-11x + 32]_{120} \\
&= -11x + 32
\end{aligned}$$

6.9. Calculating c_1

$$
\begin{aligned}
c_1 &= \left[\left\lfloor \left(\frac{t \cdot (\mathtt{ct}_1[0] \cdot \mathtt{ct}_2[1] + \mathtt{ct}_1[1] \cdot \mathtt{ct}_2[0])}{q} \right) \right\rceil\right]_q \\
&= \left[\left\lfloor \frac{(12 \cdot ((-10x + 11)(11x - 11)) + 9 \cdot (9x + 21))}{120} \right\rceil\right]_{120} \\
&= \left[\left\lfloor \frac{(12 \cdot ((231x - 11) + (81x + 189)))}{120} \right\rceil\right]_{120} \\
&= \left[\left\lfloor \frac{12 \cdot (312x + 178)}{120} \right\rceil\right]_{120} \\
&= [\lfloor 0.1 \cdot (312x + 178) \rceil]_{120} \\
&= [\lfloor 31.2x + 17.8 \rceil]_{120} \\
&= [31x + 18]_{120} \\
&= 31x + 18
\end{aligned}
$$

For the first multiplication:

$$
\begin{aligned}
(-10x + 11)(11x - 11) &= -110x^2 + 110x + 121x - 121 \\
&\equiv 110 + 231x - 121 \\
&= 231x - 11
\end{aligned}
$$

For the second multiplication:

$$9 \cdot (9x + 21) = 81x + 189$$

6.10. Calculating c_2

$$
\begin{aligned}
c_2 &= \left[\left\lfloor \left(\frac{t \cdot (\mathtt{ct}_1[1] \cdot \mathtt{ct}_2[1])}{q} \right) \right\rceil\right]_q \\
&= \left[\left\lfloor \left(\frac{12 \cdot (9 \cdot (11x - 11))}{120} \right) \right\rceil\right]_{120} \\
&= [\lfloor 0.1 \cdot (99x - 99) \rceil]_{120} \\
&= [\lfloor 9.9x - 9.9 \rceil]_{120} \\
&= [10x - 10]_{120} \\
&= 10x - 10
\end{aligned}
$$

6.11. Multiplication Ciphertext

Concluding the last three sections, one obtains the multiplication ciphertext:

$$\mathtt{Enc}(m_1 \cdot m_2) = \mathtt{ct}_X = (c_0, c_1, c_2) = ([-11x + 32], [31x + 18], [10x - 10])$$

Relinearization

Calculate:

$$(\boldsymbol{c_{2,0}}, \boldsymbol{c_{2,1}}) = \left(\left[\left\lfloor \frac{\boldsymbol{c_2} \cdot \texttt{rlk}[0]}{p} \right\rceil \right]_q, \left[\left\lfloor \frac{\boldsymbol{c_2} \cdot \texttt{rlk}[1]}{p} \right\rceil \right]_q \right)$$

Then return

$$\left(\left[\boldsymbol{c_0} + \boldsymbol{c_{2,0}} \right]_q, \left[\boldsymbol{c_1} + \boldsymbol{c_{2,1}} \right]_q \right)$$

Reduces the degree-2 ciphertext back to a standard degree-1 ciphertext, maintaining a consistent size.

Remember

$$\texttt{rlk} = ([150x - 50], 50), \texttt{ct}_X = ([-11x + 32], [31x + 18], [10x - 10])$$

6.12. Calculate Helper Tuple

To start the relinearization, calculate

$$\begin{aligned}
(\boldsymbol{c_{2,0}}, \boldsymbol{c_{2,1}}) &= \left(\left[\left\lfloor \frac{\boldsymbol{c_2} \cdot \texttt{rlk}[0]}{p} \right\rceil \right]_q, \left[\left\lfloor \frac{\boldsymbol{c_2} \cdot \texttt{rlk}[1]}{p} \right\rceil \right]_q \right) \\
\rightarrow \boldsymbol{c_{2,0}} &= \left[\left\lfloor \frac{(10x - 10)(150x - 50)}{100} \right\rceil \right]_{120} \\
&= \left[\left\lfloor \frac{1500x^2 - 500x - 1500x + 500}{100} \right\rceil \right]_{120} \\
&\equiv \left[\left\lfloor \frac{-1500 - 500x - 1500x + 500}{100} \right\rceil \right]_{120} \\
&= \left[\left\lfloor \frac{-2000x - 1000}{100} \right\rceil \right]_{120} \\
&= [\lfloor -20x - 10 \rceil]_{120} \\
&= -20x - 10 \\
\rightarrow \boldsymbol{c_{2,1}} &= \left[\left\lfloor \frac{(10x - 10) \cdot 50}{100} \right\rceil \right]_{120} \\
&= \left[\left\lfloor \frac{500x - 500}{100} \right\rceil \right]_{120} \\
&= [\lfloor 5x - 5 \rceil]_{120} \\
&= 5x - 5 \\
\Leftrightarrow (\boldsymbol{c_{2,0}}, \boldsymbol{c_{2,1}}) &= ([-20x - 10], [5x - 5])
\end{aligned}$$

Now the helper tuple can be plugged in to obtain the relinearized multiplication ciphertext.

6.13. Relinearization Output

$$\begin{aligned}\left([\boldsymbol{c_0}+\boldsymbol{c_{2,0}}]_q, [\boldsymbol{c_1}+\boldsymbol{c_{2,1}}]_q\right) &= ([(-11x+32)+(-20x-10)]_{120}, [(31x+18)+(5x-5)]_{120}) \\ &= ([-31x+22]_{120}, [36x+13]_{120}) \\ &= (\underbrace{[-31x+22]}_{\widetilde{c_0}}, \underbrace{[36x+13]}_{\widetilde{c_1}}) \\ &= \widetilde{\mathtt{ct}}\end{aligned}$$

Decryption

Take $\mathtt{sk}$ and $\mathtt{ct}$ with $\boldsymbol{c_0} = \mathtt{ct}[0]$ and $\boldsymbol{c_1} = \mathtt{ct}[1]$, then compute:

$$m = \left[\left\lfloor \frac{t \cdot [\boldsymbol{c_0} + \boldsymbol{c_1} \cdot \boldsymbol{s}]_q}{q} \right\rceil\right]_t$$

Unmasks the ciphertext and scales it back to the plaintext space, using rounding to eliminate accumulated noise.

6.14. Decrypting the relinearized ciphertext

$$\begin{aligned}\mathtt{Dec}(\widetilde{\mathtt{ct}}) &= \left[\left\lfloor \frac{t \cdot [\widetilde{\boldsymbol{c_0}} + \widetilde{\boldsymbol{c_1}} \cdot \boldsymbol{s}]_q}{q} \right\rceil\right]_t \\ &= \left[\left\lfloor \frac{12 \cdot [(-31x+22)+(36x+13)(x+1)]_{120}}{120} \right\rceil\right]_{10} \\ &= \left[\left\lfloor \frac{12 \cdot [-31x+22+49x-23]_{120}}{120} \right\rceil\right]_{10} \\ &= \left[\left\lfloor \frac{12 \cdot [18x-1]_{120}}{120} \right\rceil\right]_{10} \\ &= \left[\left\lfloor \frac{12 \cdot (18x-1)}{120} \right\rceil\right]_{10} \\ &= [\lfloor 0.1 \cdot (18x-1) \rceil]_{10} \\ &= [\lfloor 1.8x - 0.1 \rceil]_{10} \\ &= [2x]_{10} \\ &= 2x \qquad \square\end{aligned}$$

Important Note

Since the messages that were to decrypt are $m_1 = 2 \wedge m_2 = x$, the decrypted multiplication of these two correctly shows

$$2 \cdot x = m_1 \cdot m_2$$

7. Necessity of Bootstrapping

The BFV Scheme was one of the first homomorphic encryption schemes. Its proof of concept was shown in the last chapter. It is important to mention that if the random noise added exceeds a certain threshold over the course of several calculations (especially multiplication), the decryption will not restore the original messages. This works because $\Delta = \lfloor \frac{q}{t} \rfloor$ shifts the message's bits to the front while the noise is happening in the lower bits. Thus $\|e\| < \frac{\Delta}{2}$ is an essential condition for a correct decryption.

7.1. Background

> When multiplying two ciphertexts the following happens to the noise:
>
> $$(m_1 + e_1) \cdot (m_2 + e_2) = m_1 m_2 + \underbrace{m_1 e_2 + m_2 e_1 + e_1 e_2}_{\text{Noise Term}}$$

The noise term grows quadratically with each multiplication performed, meaning that the number of multiplications is limited strictly by this bound. To achieve a fully homomorphic procedure a method called Bootstrapping has to be applied to the ciphertext. It serves the purpose of reencrypting, repacking and redecrypting the ciphertext with the help of numerical tools to lower the noise and reduce its quadratic growth. The possibility of a scheme becoming fully homomorphic was first discovered by Gentry in his Dissertation.(Gentry, 2009)

Bootstrapping is very computationally heavy making endless operations on ciphertext possible in theory, but hard to efficiently compute.

After Gentry's breakthrough many other methods for bootstrapping started evolving. It is to say that for the sake of complexity, the Number Theory heavy bootstrapping procedure will be omitted and the focus of this work will lie on the somewhat homomorphic version of the BFV Scheme. More about the BFV-Bootstrapping can be read in (Gentry et al., 2011) and (Ko, 2025).

The following table shows that the bootstrapping procedure is rather slow.

Scheme	Library	Number of Slots	Bits of Precision	Latency (sec)	Throughput slots/sec	Notes
CGGI TFHE	OpenFHE	1	1	0.0295	33.9	Latency of NAND
			3-4	0.450	2.22	Latency of f(x) mod p
CKKS	OpenFHE	32,768	13	44.0	745	Full packing
			28	92.4	355	2 bootstrapping iterations to increase precision [BCC+22]
CKKS	OpenFHE	32	18	21.5	1.49	Sparse packing
			36	45.3	0.71	2 bootstrapping iterations to increase precision [BCC+22]
BGV	HElib	1,024	1	15	68.3	Thin bootstrapping [HS21]
			16	163	6.28	Full bootstrapping [HS21]

Figure 1. Comparison of bootstrapping performance on a modern CPU

This table provided in (Badawi & Polyakov, 2023) shows the computational expense of several bootstrapping algorithms.

8. Industry Usage

Homomorphic Encryption Schemes have several use cases. A few of them being within Private Information Retrieval (Kim et al., 2025), Cloud Computing, Healthcare, Financial Services and Voting.

8.1. Private Information Retrieval

Private Information Retrieval is a cryptographic protocol that enables a user to retrieve a specific item from a server that possesses a database, without revealing to the server which particular item is being requested. In other words, PIR allows the user to access information while preserving the confidentiality of the query itself, thereby preventing the database owner or service provider from learning the user's interests, intentions, or search patterns.

Historically, searching a third-party database meant exposing the query itself, which can inadvertently leak highly sensitive information. For instance, a pharmaceutical company querying a patent database could reveal its current research focus, future product strategy, or competitive priorities simply through its search terms. Similarly, individuals using online services may unintentionally disclose medical concerns, political interests, financial intentions, or geographic movements through seemingly ordinary queries. By integrating HE into PIR schemes, the user encrypts their query before sending it to the server. The server then processes this encrypted query against its database using homomorphic operations and returns an encrypted result, without ever learning which record was selected. Finally, the user decrypts the result locally on their own device. (Kim et al., 2025), (Zhuravlev, 2015)

8.2. Secure Cloud Computing

The widespread adoption of cloud computing requires organizations to outsource data storage and processing to third-party providers. However, traditional encryption only protects data at rest and in transit. Data must typically be decrypted to be processed, exposing it to potential breaches, insider threats, or compelled access on the server side.(Kiesel et al., 2023)

Homomorphic encryption fundamentally alters the cloud trust model. Organizations can upload fully homomorphically encrypted datasets to the cloud. The cloud provider can then run complex analytics, machine learning inferences, or routine data processing directly on the encrypted data. The provider remains completely blind to both the inputs and the outputs, returning an encrypted result that only the data owner can decrypt. This "zero-trust" computation model allows highly regulated industries to leverage scalable cloud infrastructure without violating data sovereignty or privacy constraints.(Kiesel et al., 2023), (Mahmoud Fahsi & Rahmani, 2015)

8.3. Healthcare

The healthcare industry sits at the intersection of immense data value and stringent privacy regulations. (Rauthan, 2025). Analyzing vast datasets of patient records, genetic profiles, and medical imaging is crucial for training diagnostic AI and accelerating drug discovery, but sharing this plaintext data across institutions is legally and ethically restricted.

HE enables secure, cross-institutional medical collaboration. Multiple hospitals can pool their data in an encrypted format into a centralized repository. Researchers can then train predictive machine learning models on this aggregated, encrypted dataset. Furthermore, HE facilitates "Privacy-Preserving Machine Learning as a Service" (PPMLaaS) (Zhang et al., 2022). A patient could send an encrypted DNA sequence to a cloud-based diagnostic model, and the model could return an encrypted prediction regarding disease susceptibility, ensuring the patient's genetic data is never exposed to the model provider.(Kocabaş & Soyata, 2015)

8.4. Financial Services and Fraud Detection

Financial institutions process a continuous flow of highly sensitive transactional data, personally identifiable information, and proprietary trading algorithms. This information is central to modern banking operations, but it also represents a significant privacy and security liability if exposed to unauthorized parties. A persistent challenge in the financial sector is detecting systemic fraud and money laundering, which often involves bad actors operating across multiple different banks, jurisdictions, and financial service providers.(Canillas et al., 2018)

Due to privacy laws, regulatory obligations, and competitive interests, banks cannot simply share their customer ledgers or internal risk models with one another. Direct data exchange would create serious confidentiality concerns and could violate legal frameworks designed to protect consumer information. HE provides a mechanism for secure multiparty computation in anti-money laundering and fraud detection by allowing institutions to collaborate on encrypted data rather than plaintext records. Banks can encrypt their transaction logs and collaboratively query them to identify suspicious patterns, repeated transaction structures, or overlapping fraudulent accounts without ever exposing their underlying client lists. This makes it possible to detect broader criminal networks while preserving institutional secrecy and customer privacy.

8.5. Secure Electronic Voting

Electronic voting systems have long struggled to balance two seemingly contradictory requirements: absolute voter privacy and end-to-end election verifiability, meaning that every eligible vote must be counted accurately and transparently. On one hand, a democratic election must protect the secrecy of each individual ballot to prevent voter coercion, retaliation, and vote-buying. On the other hand, the electoral process must remain auditable so that voters, election authorities, and independent observers can verify that the final outcome has not been manipulated.

Homomorphic encryption offers a mathematical solution to this duality by enabling encrypted votes to be processed without first decrypting them. In an HE-based voting scheme, a voter's ballot is encrypted on their local device before being submitted to the election authority's server. Because the encryption is homomorphic, the server can mathematically aggregate all encrypted ballots together to produce a final encrypted sum, without ever accessing the content of any individual vote. The election commission then uses a distributed private key, often shared among multiple trusted authorities, to decrypt only this final sum, revealing the election results while preserving the secrecy of each ballot. (Zhan et al., 2024)

9. Conclusion

The evolution of the digital landscape has created an unprecedented need for cryptographic solutions that go beyond merely securing data in transit or at rest. As explored in this work, Homomorphic Encryption presents a profound paradigm shift by enabling direct computation on encrypted information, bridging the long-standing gap between data utility and privacy. In a world where personal, financial, medical, and organizational data is increasingly outsourced to cloud infrastructures and third-party platforms, the ability to process information without exposing its plaintext form represents a major advancement in modern cryptography.

This importance becomes especially clear when considering the broader role of cryptography in contemporary digital systems. Traditional encryption methods have been highly effective in protecting stored data and securing communication channels, but they fail to address the vulnerability of "data in use." Whenever encrypted information must be decrypted for analysis, querying, or computation, it temporarily becomes exposed to the surrounding processing environment. Homomorphic Encryption directly addresses this limitation by allowing meaningful operations to be performed while the data remains encrypted. As a result, it extends the protective boundary of cryptography from storage and transmission into the computational process itself.

Through the mathematical demonstration of the BFV scheme, the mechanics of a Somewhat Homomorphic Encryption system become clear, illustrating how operations on ciphertexts are reflected in the underlying plaintexts. The BFV scheme demonstrates that encrypted addition and multiplication can preserve the correctness of the final decrypted result. However, this walkthrough also highlights the scheme's core limitation: the rapid, quadratic accumulation of noise during multiplication. While a controlled amount of noise is essential to the security of lattice-based encryption, excessive noise eventually prevents correct decryption and limits the number of computations that can be performed. This makes noise management one of the central challenges in practical Homomorphic Encryption.

Gentry's introduction of bootstrapping offers a mathematical bridge to Fully Homomorphic Encryption (FHE) by refreshing ciphertexts before their noise becomes too large. In theory, this enables an unlimited number of encrypted computations and transforms Homomorphic Encryption into a fully general framework for privacy-preserving computation. Nevertheless, the computational expense of repacking, refreshing, and re-evaluating ciphertexts remains the primary bottleneck preventing widespread adoption. Even though major improvements have been made in scheme design, parameter selection, batching, and hardware acceleration, FHE still requires significantly more computational resources than conventional plaintext computation.

Despite these performance challenges, the conceptual breakthrough is undeniable and is already relevant across highly regulated sectors. In Private Information Retrieval, Homomorphic Encryption allows users to query databases without revealing what they are searching for. In financial systems, it enables banks and credit agencies to collaborate on fraud detection, anti-money laundering analysis, and credit scoring without exposing raw customer records. In healthcare and cloud computing, it offers the possibility of analyzing sensitive medical or organizational data while maintaining confidentiality. Similarly, in secure electronic voting, HE provides a mechanism for combining ballot secrecy with verifiable tallying, ensuring that democratic processes can remain both private and auditable.

These applications demonstrate that Homomorphic Encryption is not merely a theoretical advancement, but a response to a practical and increasingly urgent security problem. As societies become more dependent on data-driven decision-making, artificial intelligence, and distributed computation, the demand for privacy-preserving technologies will continue to intensify. The future relevance of HE will therefore depend not only on mathematical correctness, but also on its ability to become efficient, scalable, and accessible enough for real-world deployment.

References

Aaraj, N. (2022). *Survey on Fully Homomorphic Encryption, Theory, and Applications.*

Acar, A., AKSU, H., ULUAGAC, A. S., & CONTI, M. (2017). *A Survey on Homomorphic Encryption Schemes: Theory and Implementation.*

Badawi, A. A., & Polyakov, Y. (2023). *Demystifying Bootstrapping in Fully Homomorphic Encryption.*

Canillas, R., Rania, T., Bouchenak, S., Hasan, O., Brunie, L., & Sarrat, L. (2018). *Exploratory Study of Privacy Preserving Fraud Detection.* 25–31. https://doi.org/10.1145/3284028.3284032

Fan, J., & Vercauteren, F. (2012). *Somewhat Practical Fully Homomorphic Encryption.*

Gentry, C. (2009). *Fully homomorphic encryption using ideal lattices.*

Gentry, C., Halevi, S., & Smart, N. (2011). *Better Bootstrapping in Fully Homomorphic Encryption.*

Katz, J., & Lindell, Y. (2015). *Introduction to Modern Cryptography.*

Kiesel, R., Lakatsch, M., Mann, A., Lossie, K., Sohnius, F., & Schmitt, R. H. (2023). Potential of Homomorphic Encryption for Cloud Computing Use Cases in Manufacturing. *Journal of Cybersecurity and Privacy*, *3*(1), 44–60. https://doi.org/10.3390/jcp3010004

Kim, S., Ji, H., Kim, J., Choi, W., Park, J., & Ahn‡, J. H. (2025). *IVE: An Accelerator for Single-Server Private Information Retrieval Using Versatile Processing Elements.*

Ko, R. (2025). *The Mathematical Construction of the BFV Scheme.*

Kocabaş, Ö., & Soyata, T. (2015). *Medical Data Analytics in the Cloud Using Homomorphic Encryption* (pp. 751–768). https://doi.org/10.4018/978-1-4666-8756-1.ch038

Mahmoud Fahsi, S. M. B., & Rahmani, A. (2015). *A Framework for Homomorphic, Private Information Retrieval Protocols in the Cloud.* 16–23.

Marcolla, C., Sucasas, V., Manzano, M., Bassoli, R., & Fitzek, F. H. (2022). *Survey on Fully Homomorphic Encryption, Theory, and Applications.*

Martins, P., & others. (2017). *A Survey on Fully Homomorphic Encryption: An Engineering Perspective.*

Rauthan, J. S. (2025). *Homomorphic Encryption in Healthcare Industry Applications for Protecting Data Privacy.*

Zhan, Y., Zhao, W., Zhu, C., Zhao, Z., Yang, N., & Wang, B. (2024). Efficient Electronic Voting System Based on Homomorphic Encryption. *Electronics*, *13*(2). https://doi.org/10.3390/electronics13020286

Zhang, Q., Xiang, T., Cai, Y., Zhao, Z., Wang, N., & Wu, H. (2022). Privacy-Preserving Machine Learning as a Service: Challenges and Opportunities. *IEEE Network*, *PP*, 1–8. https://doi.org/10.1109/MNET.127.2200342

Zhuravlev, D. (2015). *Towards practical private information retrieval from homomorphic encryption.*